WEEK – 13

Experiment No. 13: Collection Framework

Title

Letter Combinations of a Phone Number Using Java Collection Framework

Objective / Aim

To write a Java program to generate all possible letter combinations corresponding to the digits of a phone number using the Java Collection Framework and demonstrate the use of the `List` interface.

Problem Statement

Given a string containing digits from 2 to 9 inclusive, return all possible letter combinations that the number could represent.

A mapping of digits to letters, similar to the keys on a telephone keypad, is given below. The digit 1 does not map to any letters.

Example:

Input:

```
23
```

Output:

```
[ad, ae, af, bd, be, bf, cd, ce, cf]
```

Explanation:

The digit 2 maps to the letters `a`, `b`, and `c`.

The digit 3 maps to the letters `d`, `e`, and `f`.

Combining each letter corresponding to digit 2 with each letter corresponding to digit 3 produces nine possible combinations:

`ad`, `ae`, `af`, `bd`, `be`, `bf`, `cd`, `ce`, `cf`.

Concepts Used

Concept	Description
Collection Framework	A set of classes and interfaces used to store and manipulate groups of objects.
List Interface	An ordered collection that allows duplicate elements.
ArrayList	A resizable array implementation of the `List` interface.
String	Used to store the input digits and letter combinations.
Array	Used to store the letter mappings for digits 2 through 9.
Iteration	Used to traverse the digits and generate letter combinations.
Nested Loops	Used to combine letters corresponding to different digits.

Classes Used

1. **LetterCombinations** – Contains the logic to generate all possible letter combinations.
2. **LetterCombinationsDemo** – Contains the `main()` method, accepts the input, and displays the resulting combinations.

Telephone Keypad Mapping

The following mapping is used in the program:

Digit	Letters
2	ABC
3	DEF
4	GHI
5	JKL
6	MNO
7	PQRS

Digit	Letters
8	TUV
9	WXYZ

The digits 0 and 1 do not have letter mappings in this program.

Algorithm / Logic

1. Start the program.
2. Define a mapping of digits 2 through 9 to their corresponding letters.
3. Read a string containing digits from 2 to 9.
4. Create an empty `List` to store the letter combinations.
5. If the input string is empty, return an empty list.
6. Initialize the list with an empty string.
7. Process each digit in the input string.
8. Find the corresponding letters for the current digit.
9. Create new combinations by appending each corresponding letter to every existing combination.
10. Store the newly generated combinations in the list.
11. Repeat the process until all digits have been processed.
12. Display all possible letter combinations.
13. Stop the program.

Source Code

File Name: LetterCombinationsDemo.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class LetterCombinations {

    private static final String[] mapping = {
        "",     // 0
        "",     // 1
        "abc",  // 2
        "def",  // 3
        "ghi",  // 4
        "jkl",  // 5
        "mno",  // 6
        "pqrs", // 7
        "tuv",  // 8
        "wxyz"  // 9
    };
```

```
    public List<String> letterCombinations(String digits) {

        List<String> result = new ArrayList<>();

        if (digits == null || digits.isEmpty()) {
            return result;
        }

        result.add("");

        for (int i = 0; i < digits.length(); i++) {

            char digit = digits.charAt(i);

            if (digit < '2' || digit > '9') {
                throw new IllegalArgumentException(
                    "Input must contain digits from 2 to 9 only."
                );
            }

            String letters = mapping[digit - '0'];

            List<String> newResult = new ArrayList<>();

            for (String combination : result) {

                for (int j = 0; j < letters.length(); j++) {

                    newResult.add(
                        combination + letters.charAt(j)
                    );
                }
            }

            result = newResult;
        }

        return result;
    }
}

public class LetterCombinationsDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            System.out.print("Enter the digit string: ");
            String digits = sc.nextLine();

            LetterCombinations obj = new LetterCombinations();

            List<String> result =
                obj.letterCombinations(digits);
```

```
            System.out.println("Letter combinations: " + result);

        } catch (IllegalArgumentException e) {

            System.out.println("Invalid input: " + e.getMessage());

        } finally {

            sc.close();
        }
    }
}
```

Explanation of the Program

1. Digit-to-Letter Mapping

The `mapping` array stores the letters corresponding to each digit. For example, `mapping[2]` contains `"abc"` and `mapping[3]` contains `"def"`.

2. List Interface

The program uses the `List<String>` interface to store the generated combinations.

```
List<String> result = new ArrayList<>();
```

The `ArrayList` class provides a resizable collection for storing the combinations.

3. Empty Input Handling

If the input string is empty, the method returns an empty list because there are no digits to process.

4. Generating Combinations

The program processes each digit in the input string. For every digit, it combines the corresponding letters with all previously generated combinations.

For the input `23`:

Initially, the list contains an empty string:

```
[""]
```

After processing digit 2:

```
[a, b, c]
```

After processing digit 3:

```
[ad, ae, af, bd, be, bf, cd, ce, cf]
```

5. Displaying the Result

The `main()` method calls `letterCombinations()` and prints the resulting list.

Compilation and Execution

Step 1: Save the program

Save the source code as:

```
LetterCombinationsDemo.java
```

Step 2: Compile the program

Open the terminal in the directory containing the file and execute:

```
javac LetterCombinationsDemo.java
```

Step 3: Run the program

```
java LetterCombinationsDemo
```

Sample Output

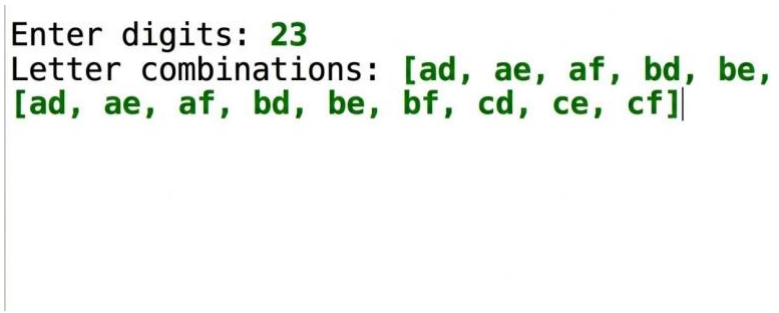
Sample Test Case 1

Input:

```
23
```

Output:

```
Enter the digit string: 23
Letter combinations: [ad, ae, af, bd, be, bf, cd, ce, cf]
```



Test Cases

Test Case 1: Two Digits

Input:

```
23
```

Expected Output:

```
[ad, ae, af, bd, be, bf, cd, ce, cf]
```

Explanation: Each of the three letters corresponding to digit 2 is combined with each of the three letters corresponding to digit 3, producing nine combinations.

Test Case 2: Single Digit

Input:

```
2
```

Expected Output:

```
[a, b, c]
```

Explanation: Digit 2 corresponds to the letters `a`, `b`, and `c`.

Test Case 3: Three Digits

Input:

```
234
```

Expected Output:

```
[adg, adh, adi, aeg, aeh, aei, afg, afh, afi, bdg, bdh, bdi, beg, beh, bei,
bfg, bfh, bfi, cdg, cdh, cdi, ceg, ceh, cei, cfg, cfh, cfi]
```

Explanation: The program generates all combinations using the letters corresponding to digits 2, 3, and 4. Since each digit has three letters, the total number of combinations is $3 \times 3 \times 3 = 27$.

Test Case 4: Four Digits

Input:

```
79
```

Expected Output:

```
[pw, px, py, pz, qw, qx, qy, qz, rw, rx, ry, rz, sw, sx, sy, sz]
```

Explanation: Digit 7 corresponds to four letters (`p`, `q`, `r`, `s`) and digit 9 corresponds to four letters (`w`, `x`, `y`, `z`). Therefore, 4 × 4 = 16 combinations are generated.

Test Case 5: Empty Input

Input:

Expected Output:

```
[]
```

Explanation: An empty input string produces an empty list of combinations.

Test Case 6: Invalid Input

Input:

```
21
```

Expected Output:

```
Invalid input: Input must contain digits from 2 to 9 only.
```

Explanation: The digit 1 does not map to any letters, so the program rejects the input.

Test Case Summary

Test Case	Input	Number of Combinations
1	`23`	9
2	`2`	3
3	`234`	27
4	`79`	16